

Computing Platform for Autonomous Driving (CP4AD)

3. RTOS #3

Prof. Jong-Chan Kim

Dept. Automobile and IT Convergence



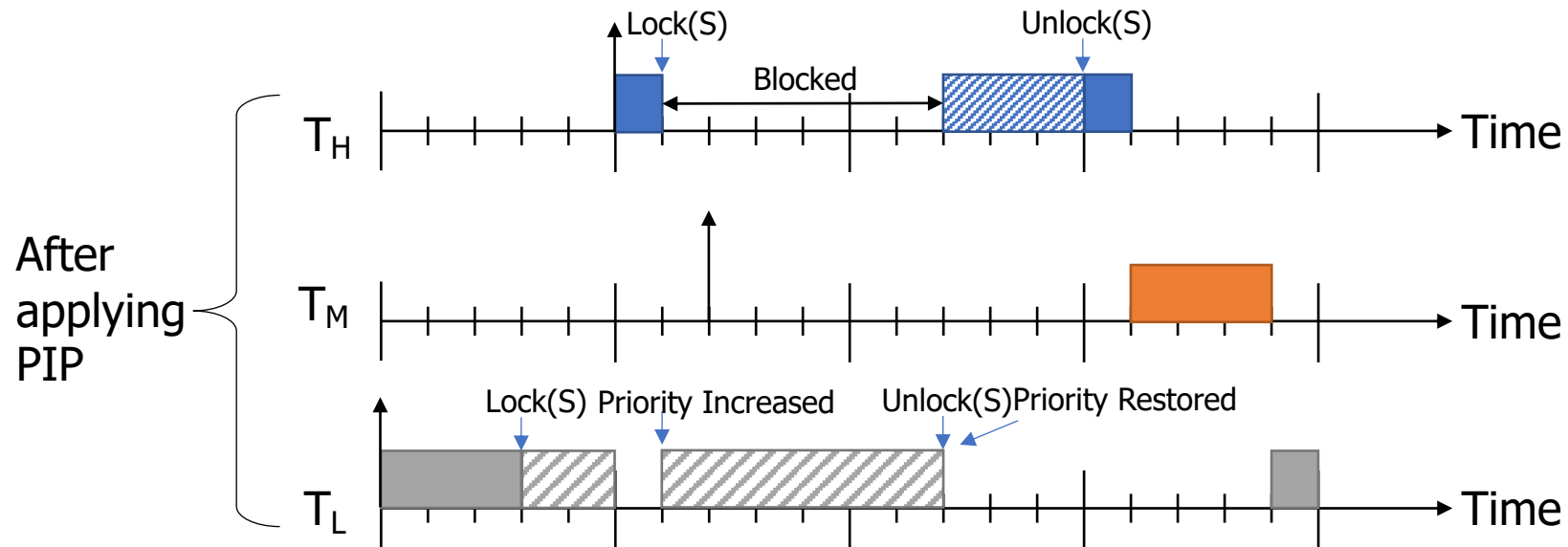
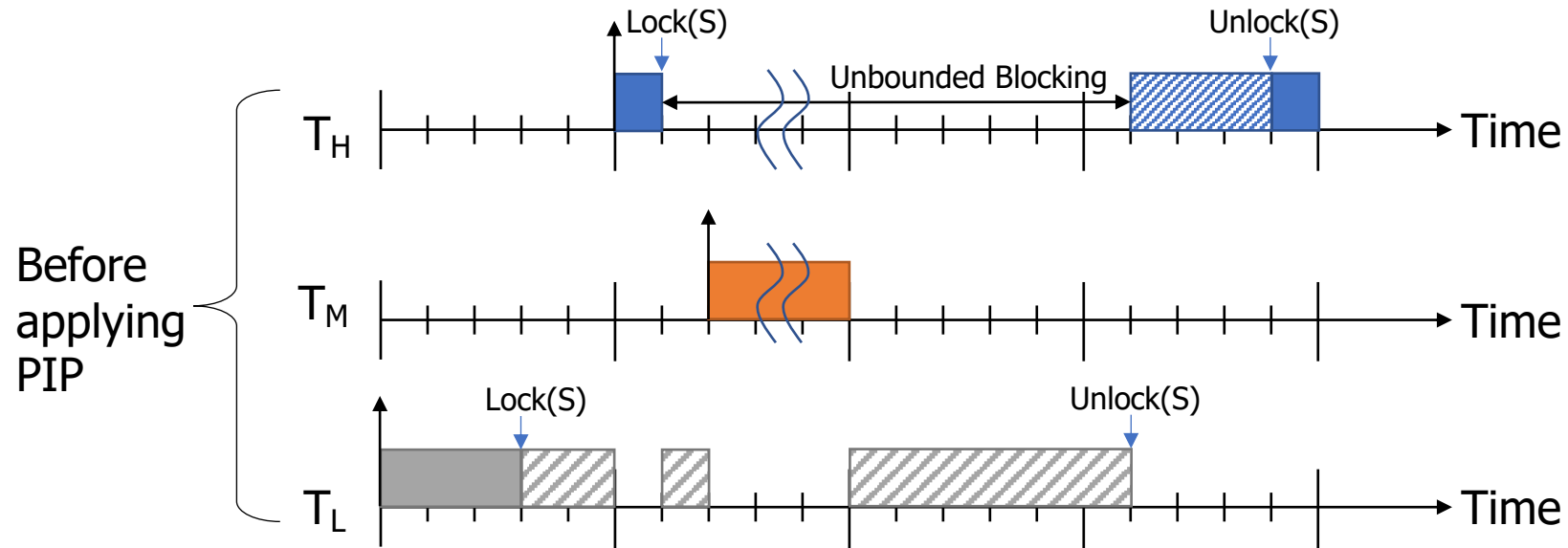
국민대학교
KOOKMIN UNIVERSITY

Resource Sharing Protocols

- PIP (Priority Inheritance Protocol)
 - When a higher priority task is blocked by a lower priority task, the blocked task's priority is inherited to the blocking task
- PCP (Priority Ceiling Protocol)
 - Tasks with higher priorities than the system ceiling are allowed to acquire locks
 - System ceiling: the highest priority among the tasks that possibly use the currently locked resources (the information of tasks that use each lock is known in advance)
 - When a task is blocked, its priority is inherited to the blocking task (PIP)
- IPCP (Immediate Priority Ceiling Protocol)
 - When locking a resource, the task's priority is immediately raised to the ceiling of the lock

The highest priority among the tasks that use the lock

Priority Inheritance Protocol (PIP)



Limitation of PIP

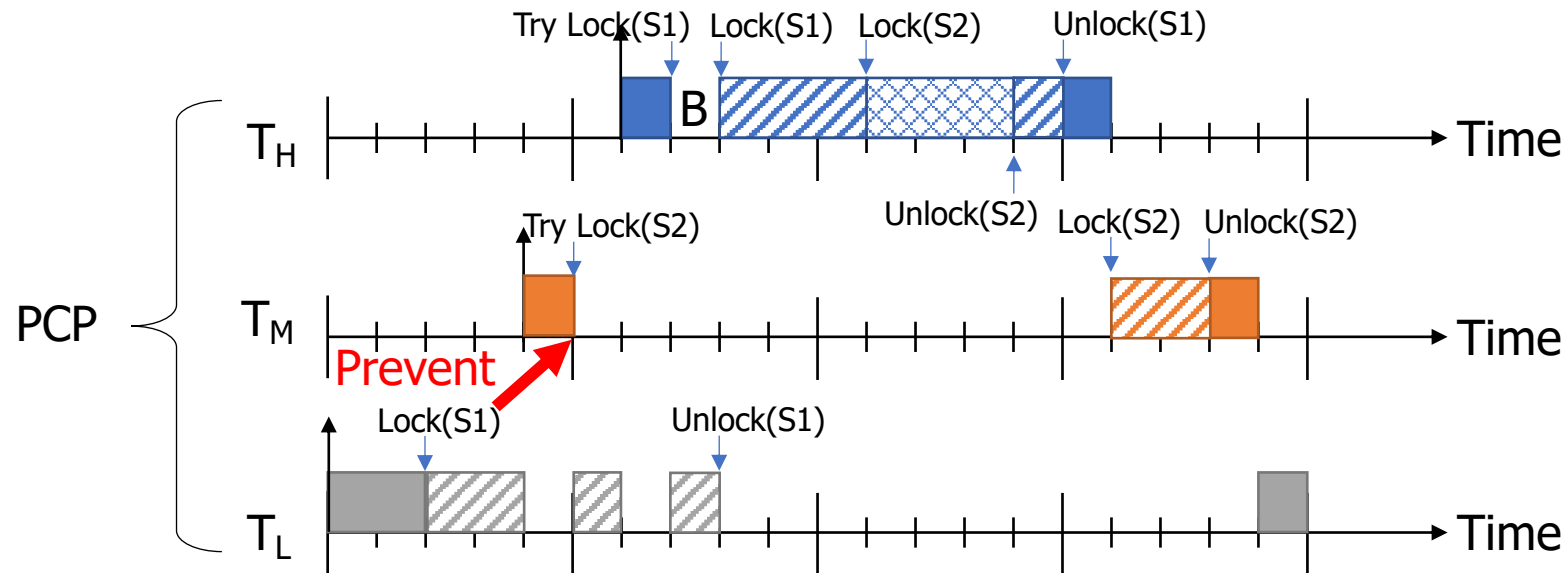
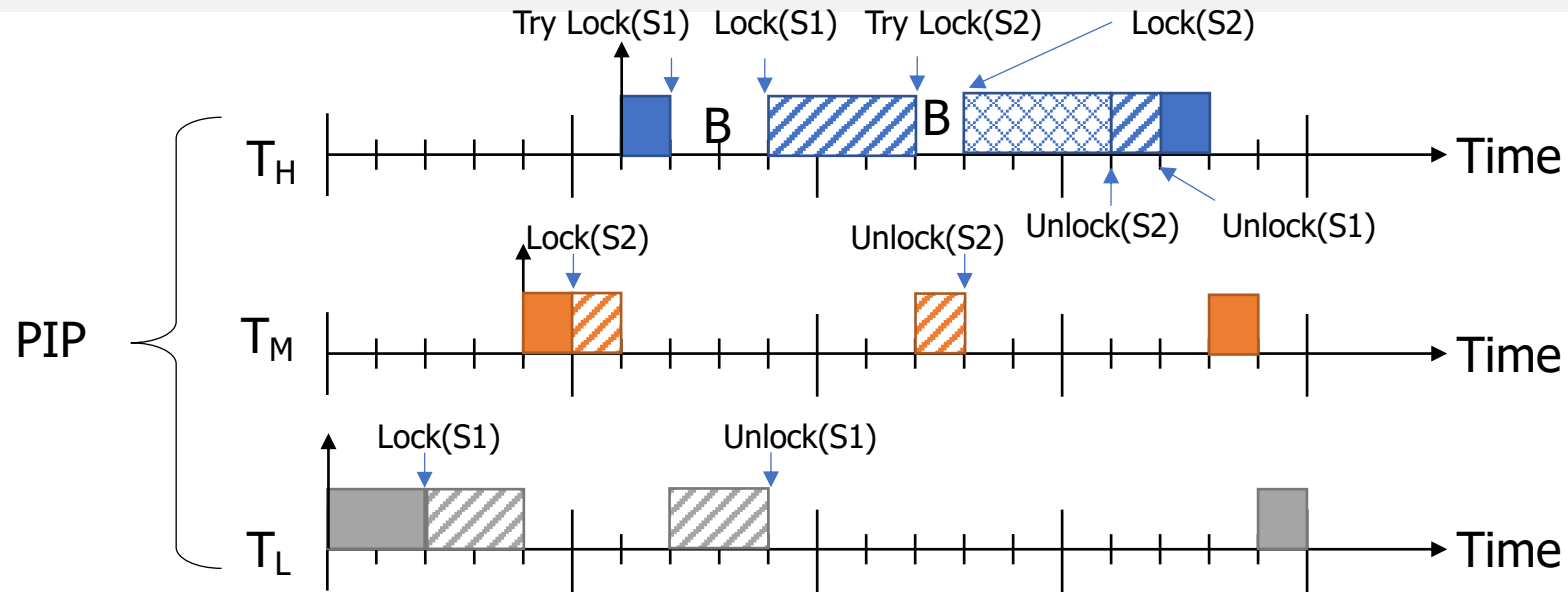
- Chained Blocking
 - When using nested locks, multiple blockings occur
- Deadlock
 - Circular wait causes deadlock

Resource Sharing Protocols

- PIP (Priority Inheritance Protocol)
 - When a higher priority task is blocked by a lower priority task, the blocked task's priority is inherited to the blocking task
- PCP (Priority Ceiling Protocol)
 - Tasks with higher priorities than the system ceiling are allowed to acquire locks
 - System ceiling: the highest priority among the tasks that possibly use the currently locked resources (the information of tasks that use each lock is known in advance)
 - When a task is blocked, its priority is inherited to the blocking task (PIP)
- IPCP (Immediate Priority Ceiling Protocol)
 - When locking a resource, the task's priority is immediately raised to the ceiling of the lock

The highest priority among the tasks that use the lock

(Original) Priority Ceiling Protocol (PCP)



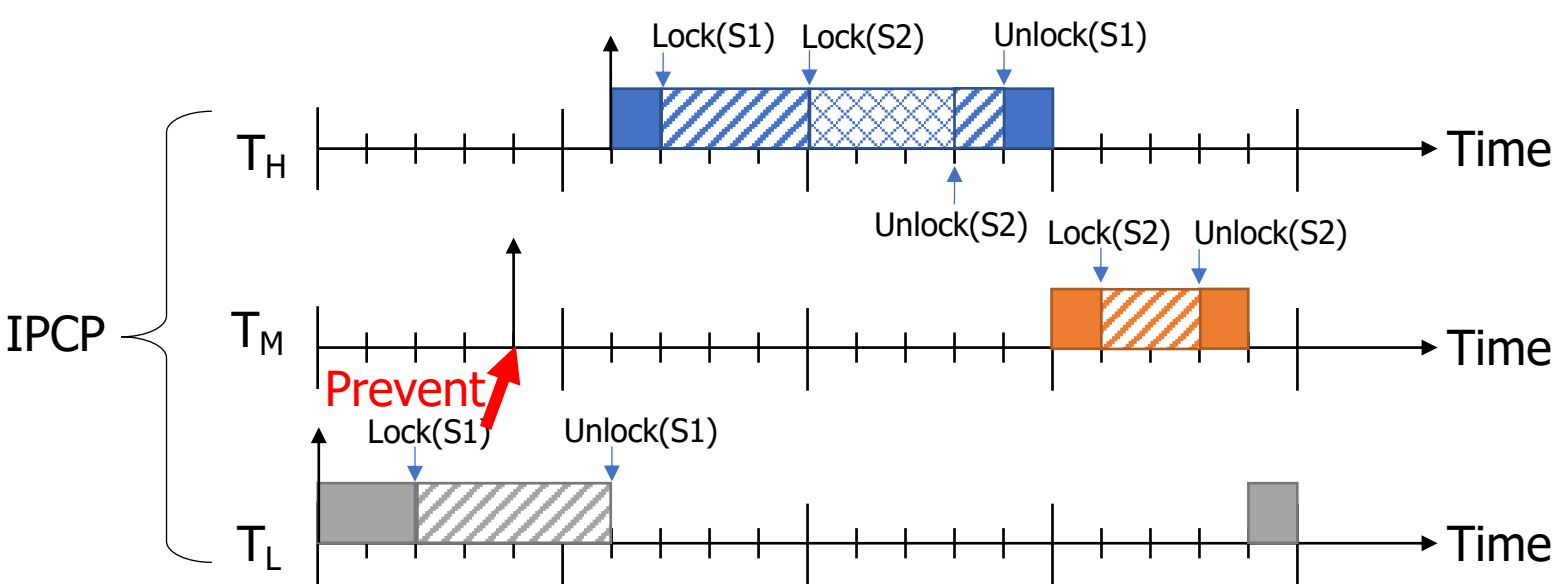
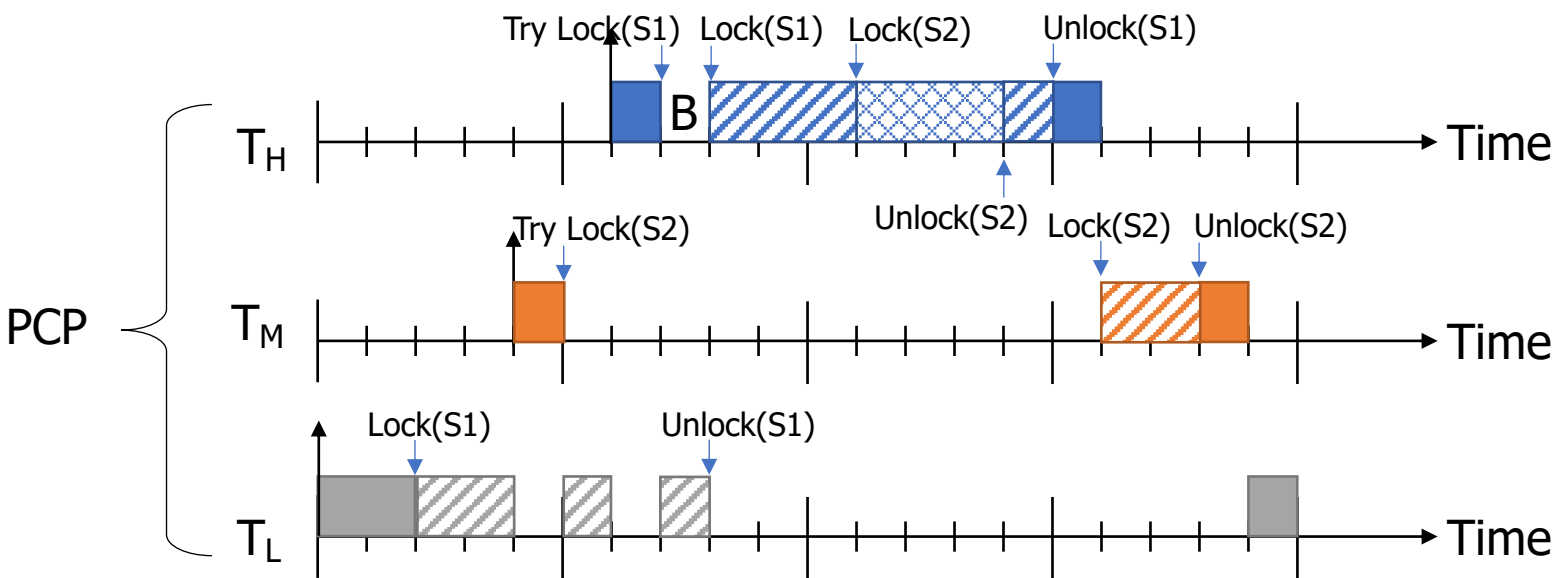
Lock(S1) prevents T_M from acquiring S2 by increasing the priority ceiling.

Resource Sharing Protocols

- PIP (Priority Inheritance Protocol)
 - When a higher priority task is blocked by a lower priority task, the blocked task's priority is inherited to the blocking task
- PCP (Priority Ceiling Protocol)
 - Tasks with higher priorities than the system ceiling are allowed to acquire locks
 - System ceiling: the highest priority among the tasks that possibly use the currently locked resources (the information of tasks that use each lock is known in advance)
 - When a task is blocked, its priority is inherited to the blocking task (PIP)
- IPCP (Immediate Priority Ceiling Protocol)
 - When locking a resource, the task's priority is immediately raised to the ceiling of the lock

The highest priority among the tasks that use the lock

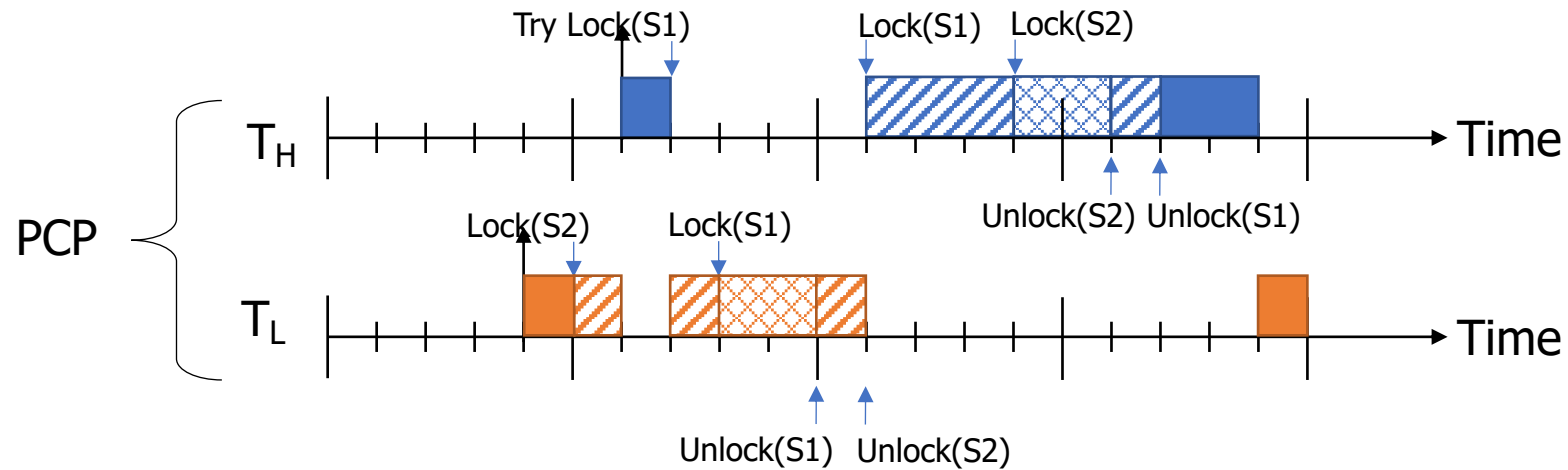
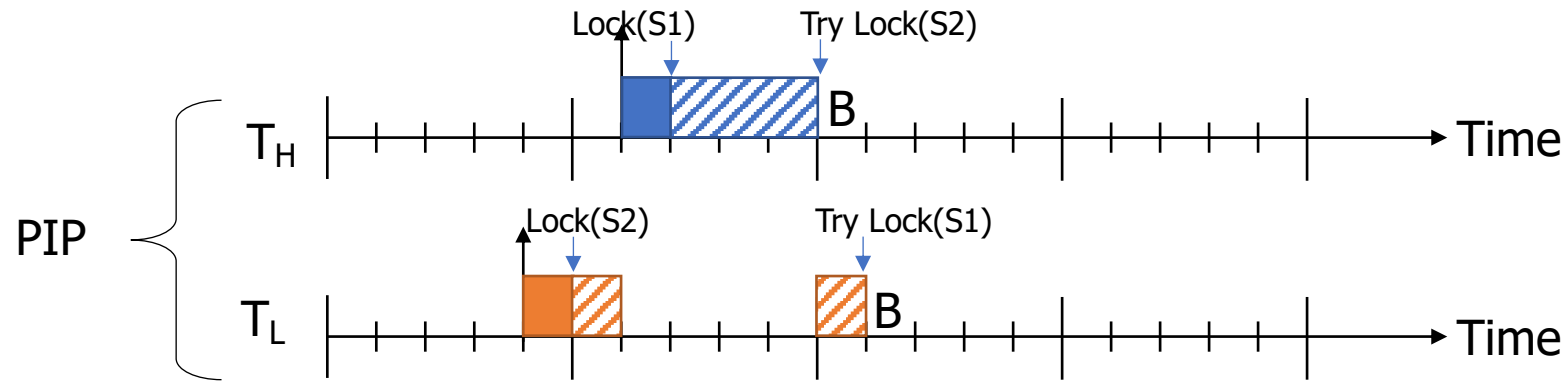
Immediate Priority Ceiling Protocol (IPCP)



T_L 's priority is immediately increased upon acquiring the lock

Lock(S_1) prevents T_M from executing by increasing T_L 's priority to T_H .

PCP for Deadlock Prevention



Deadlock is prevented by PCP by not allowing T_H to lock S1

AUTOSAR OS

- Just a part in the full AUTOSAR standards
- An extension of OSEK RTOS
- Extended features depending on the Scalability Classes (SC)
 - Schedule table
 - Timing protection
 - Memory protection



Alarm 기반 Activation의 문제점

- OSEK에서 Periodic Task 작성 방법
 - Counter와 연결된 Alarm에서 Activate하도록 OIL 설정
- 문제점
 - Alarm은 런타임에 취소/변경이 가능
 - 실수 혹은 악의적으로 Alarm 변경시 Periodic Task 오동작 가능성
- Example)
 - CancelAlarm() 호출

13.6.3.5 CancelAlarm

Syntax: StatusType CancelAlarm (AlarmType <AlarmID>)

Parameter (In):
 AlarmID Reference to an alarm

Parameter (Out): none

Description: The system service cancels the alarm <AlarmID>.

Particularities: Allowed on task level and in ISR, but not in hook routines.
Status:

- Standard:
- No error, E_OK
 - Alarm <AlarmID> not in use, E_OS_NOFUNC

- Extended:
- Alarm <AlarmID> is invalid, E_OS_ID

Conformance: BCC1, BCC2, ECC1, ECC2

Schedule Table OIL Example

```
SCHEDULETABLE SchedTab1 {
```

```
  COUNTER = counter1;
```

```
  DURATION = 10;
```

```
  REPEATING = TRUE;
```

```
  AUTOSTART = TRUE {
```

```
    START_VALUE = 5;
```

```
  };
```

```
  EXPIRE_POINT = ACTION {
```

```
    EXPIRE_VALUE = 0;
```

```
    ACTION = ACTIVATETASK { TASK = TaskH; };
```

```
    ACTION = ACTIVATETASK { TASK = TaskL; };
```

```
  };
```

```
  EXPIRE_POINT = ACTION {
```

```
    EXPIRE_VALUE = 5;
```

```
    ACTION = ACTIVATETASK { TASK = TaskH; };
```

```
    ACTION = SETEVENT { TASK = TaskL; EVENT = Event1; };
```

```
  };
```

```
};
```

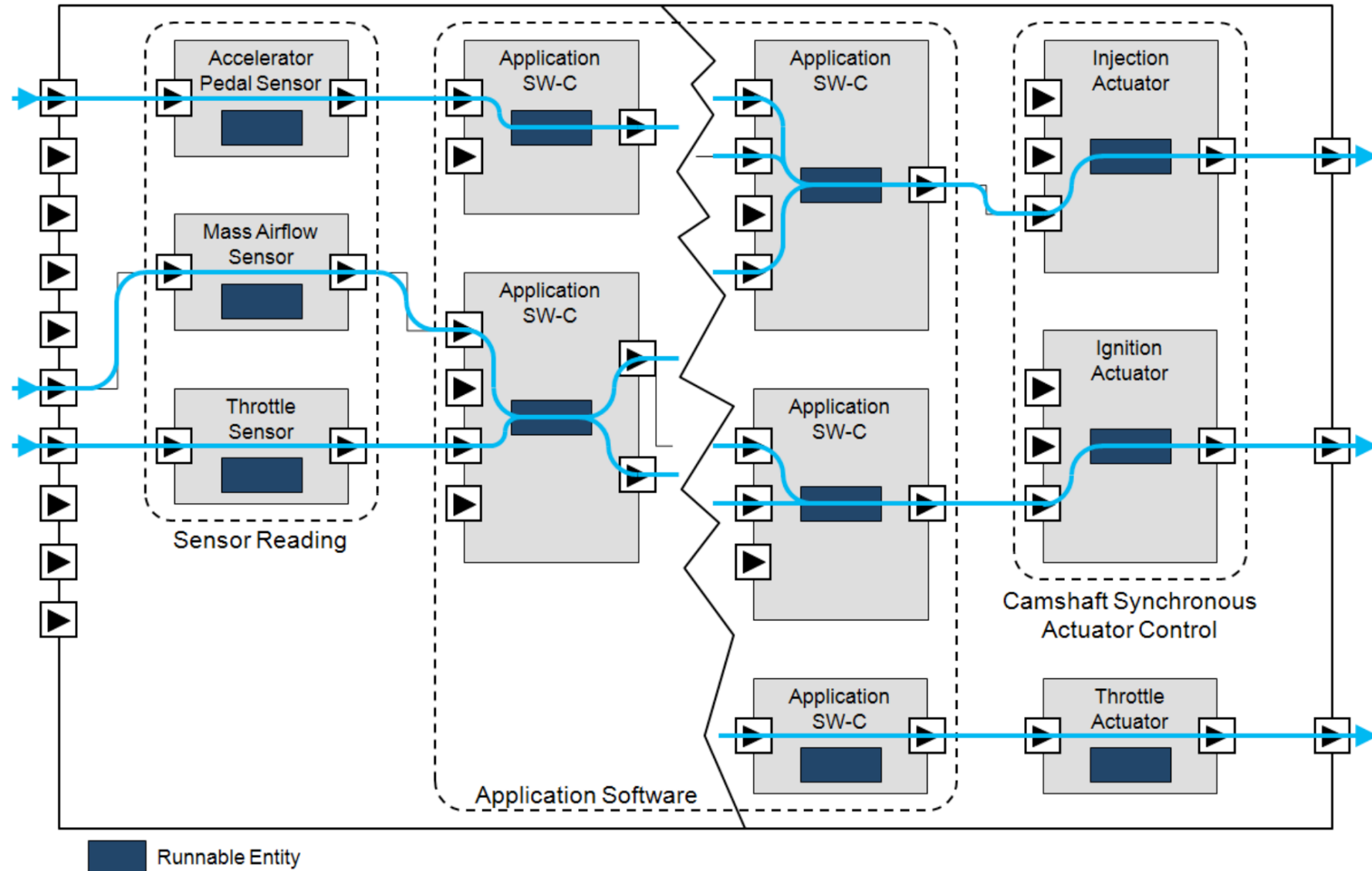
Cycle Time

Start Counter
Value

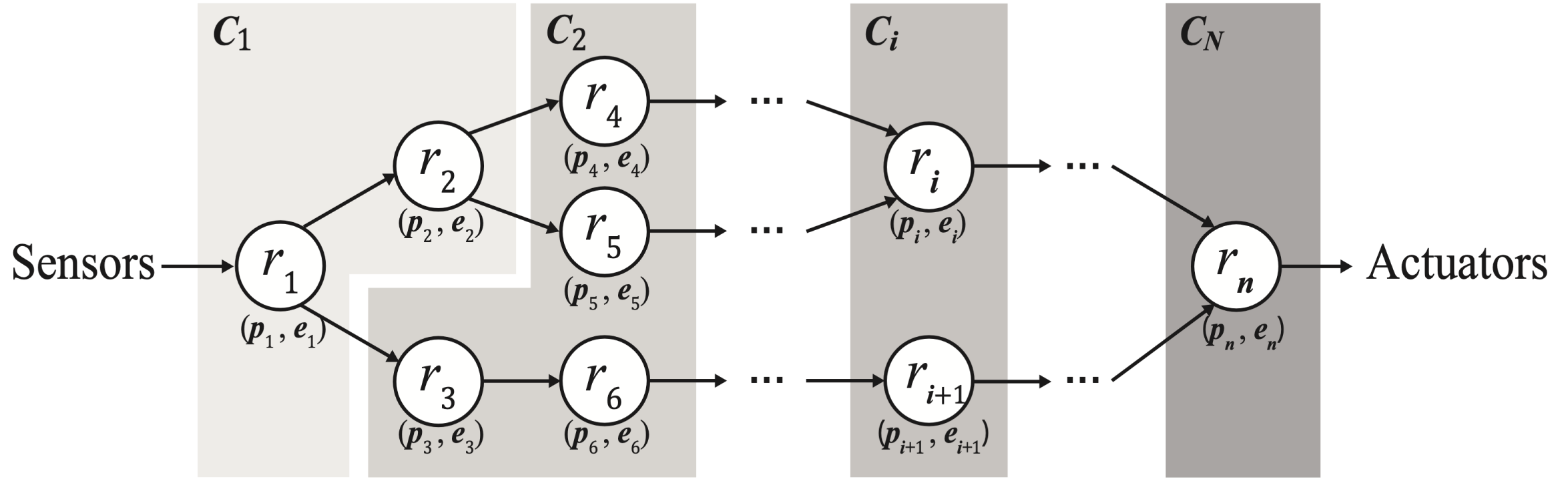
Task Activation

Event Setting

AUTOSAR-based Engine Control Model



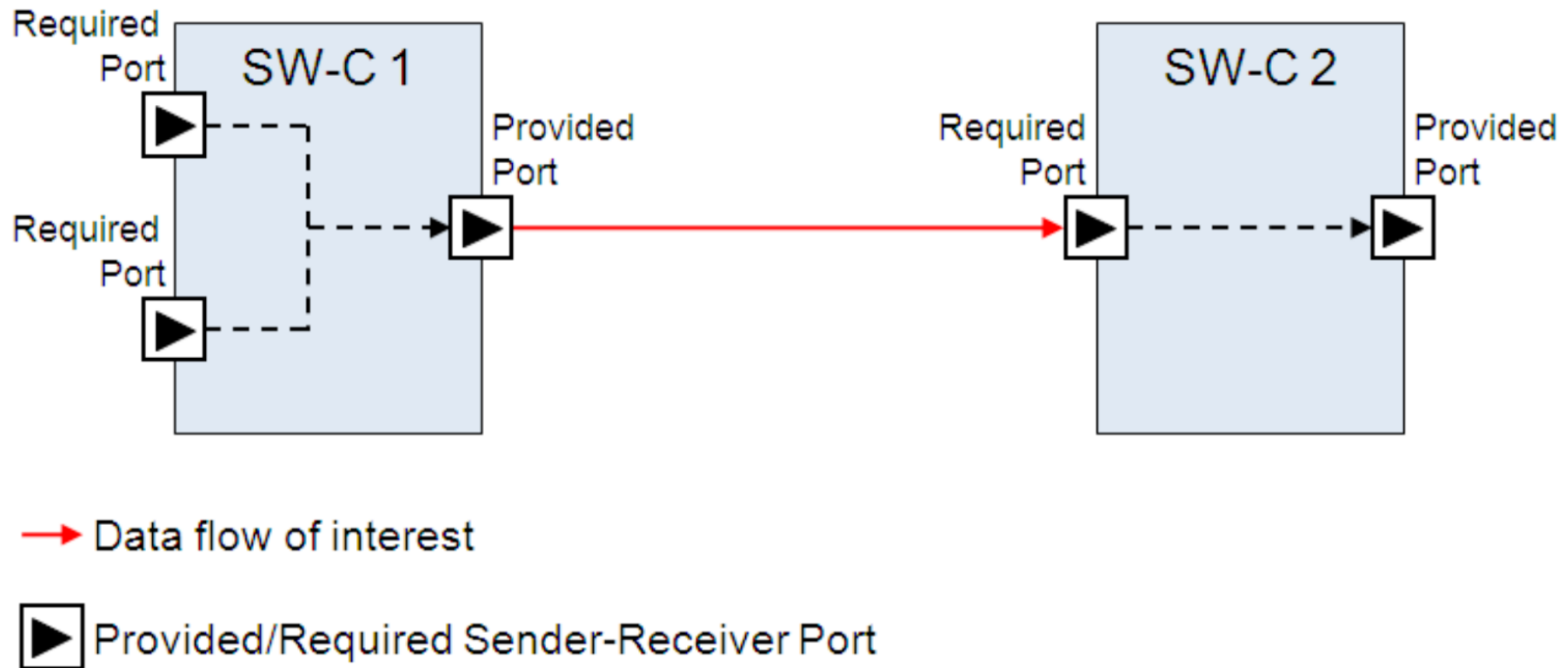
Generalized AUTOSAR Component Model



- C_i : SW Component
- r_i : Runnable

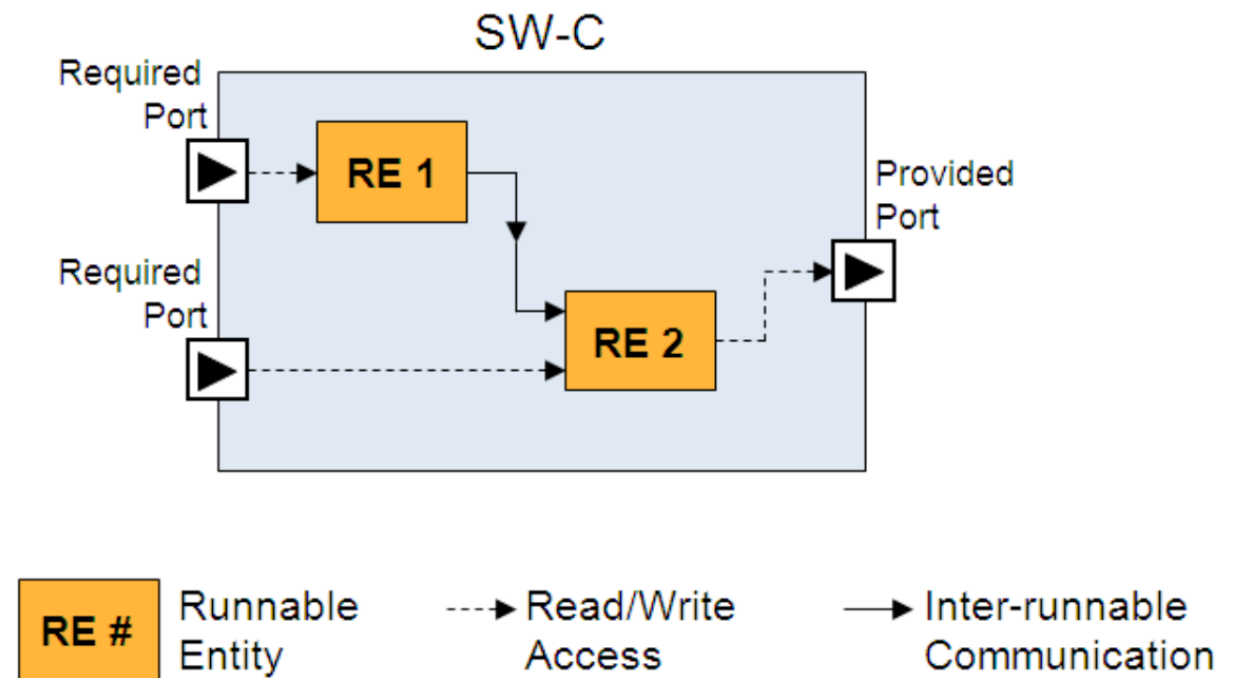
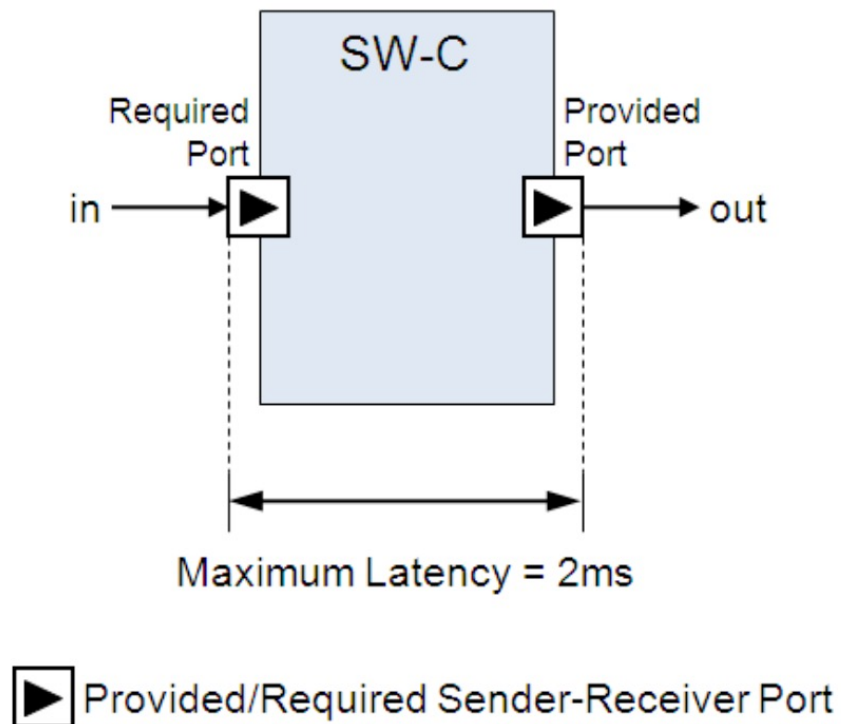
SW Component (SW-C)

- Functional unit of software design, packaging, and deployment
- Port: R-ports, P-ports for communication



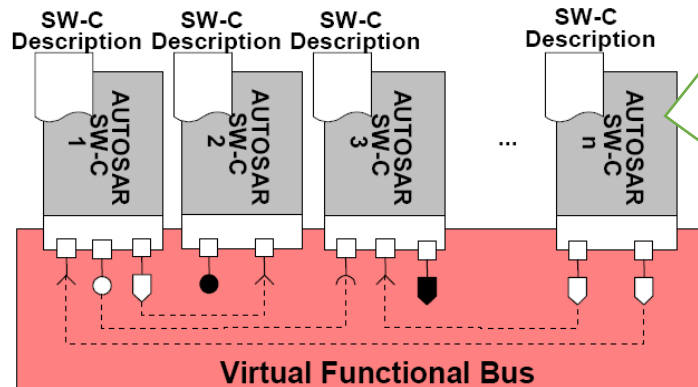
SW-C Delay Model

- How end-to-end delays are determined from sensing to actuation?



Inside a SW-C

SW-C = Set of Runnables



Period = 5ms

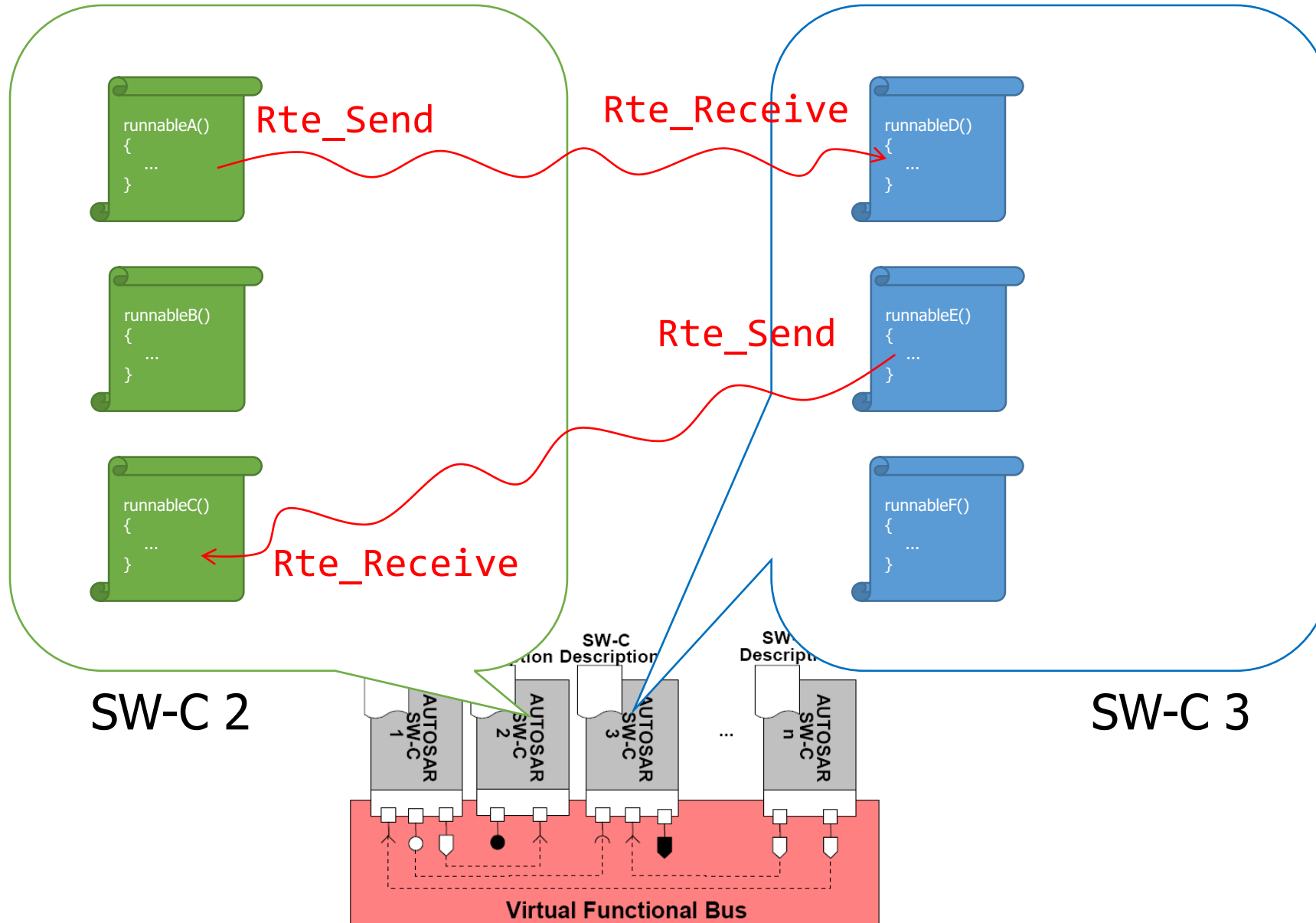


Period = 10ms



Period = 100ms

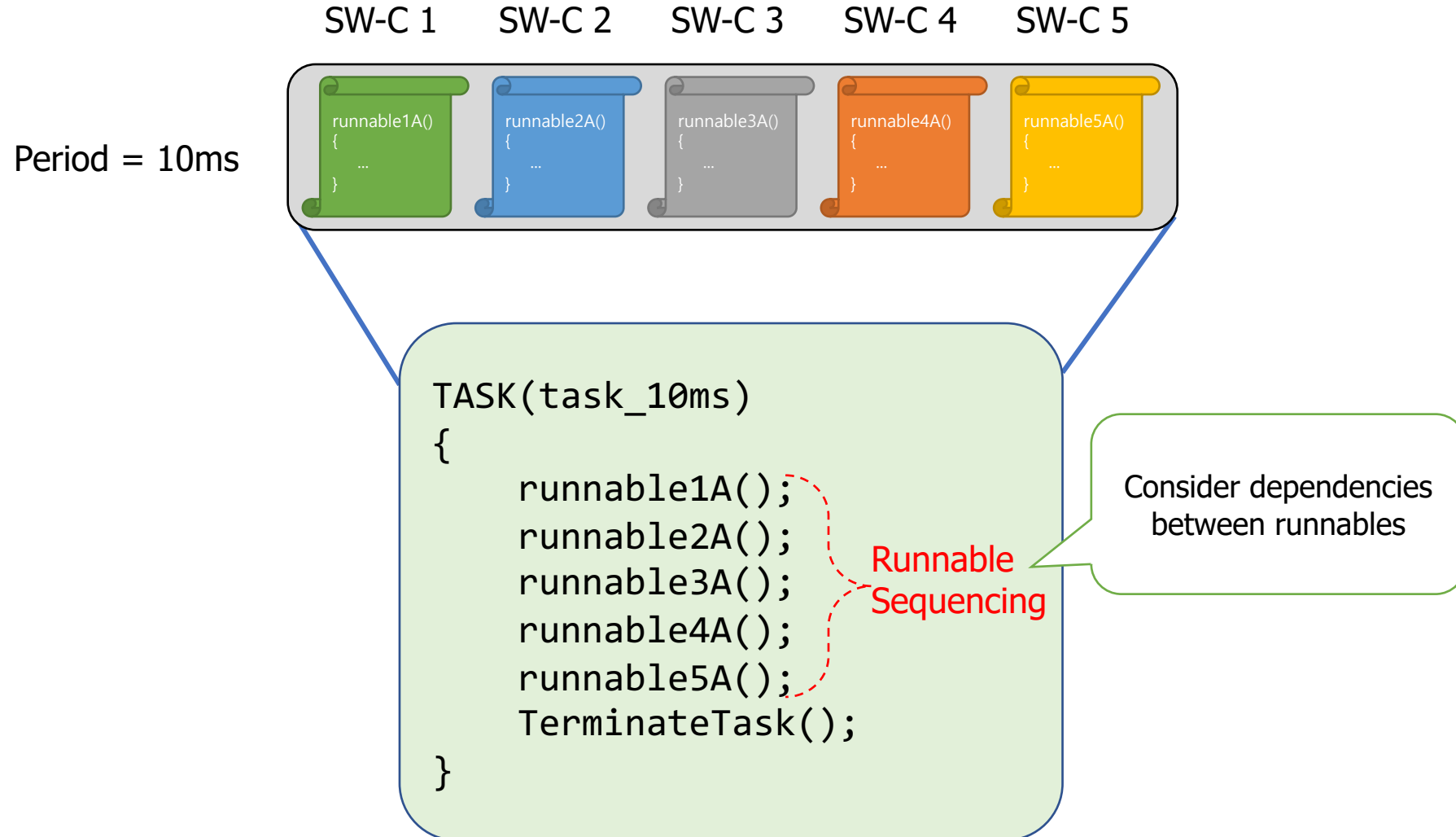
Between SW-Cs



Runnable to Task Mapping



Inside a Task



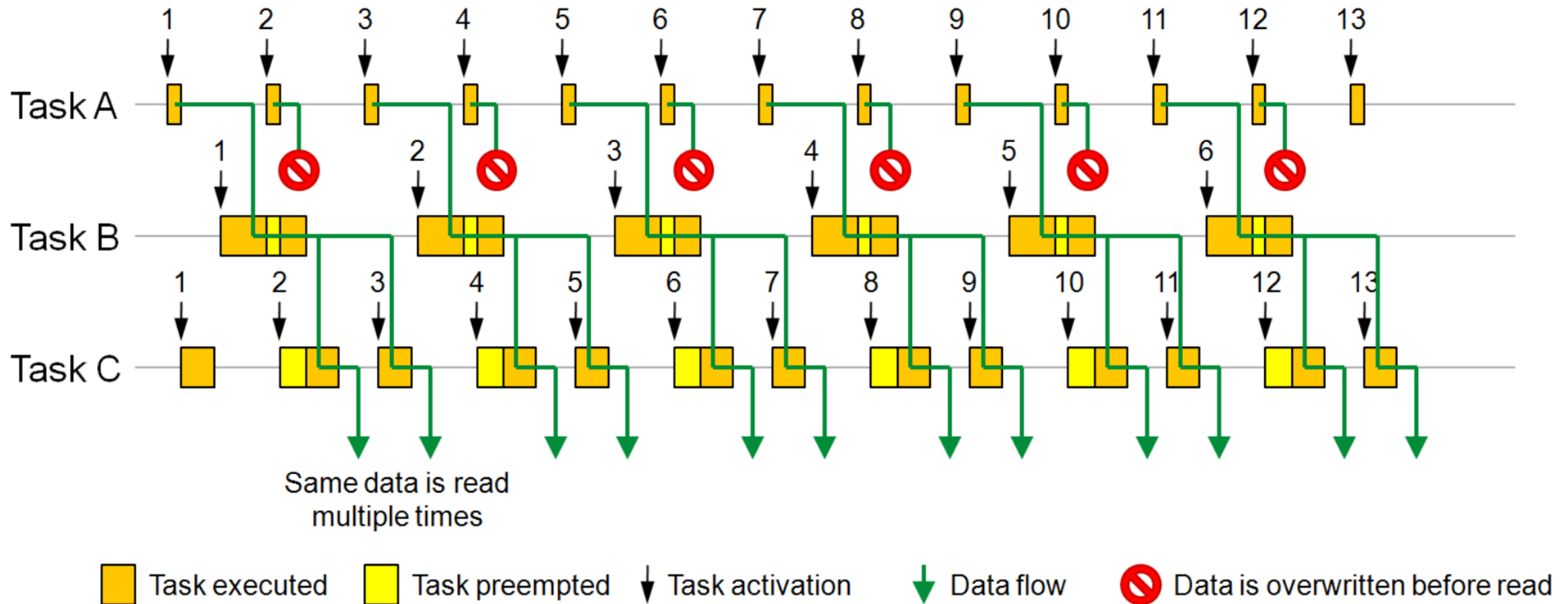
Data Age vs. Reaction Time

The *data age timing constraint* is mainly important in control engineering, but may appear in all domains. Here the focus is from the response perspective rather than from the stimulus perspective. In other words, the assumption is that last is best, i.e., it is accepted/tolerated that a value is overwritten along the path from stimulus to response. When for example an actuator value is periodically updated, it is of importance that the corresponding input values are not too old. In this case the constrained time of importance is the delay from the latest stimulus to a given response.

The *reaction time constraint* is utilized when the first reaction to a stimulus is of importance. This is usually the case in body electronics, but may also be the case in other domains. One example is the time it takes from a button is pressed to the light is switched on. Another example, from the chassis domain, is the time from the brake pedal is pressed until the brakes are activated. In both cases the constrained time of importance is the delay from a given stimulus to the first corresponding response.

Distributed Cause-Effect Chains

- End-to-end timing analysis is required



Questions

